

Tute (Week 9)

David Gries' coffee beans

A bag of coffee beans contains beans which are either black or white. Take any two beans from the bag at random: if they have different colours, throw away the white bean, but put the black bean back into the bag. If on the other hand they have the same colour, throw away both beans; but then add (back) a new white bean into the bag (from our infinite supply of extra white beans on the side). Repeat this process until there's only one bean left.

1. Suppose you know how many beans there are of each colour initially. From that, can you predict the colour of the last bean?
2. Write down a program in the language $\square^+$ that models this process by updating two variables b (number of black beans) and w (number of white beans). Use the non-deterministic choice operator $+$ to model the act of picking two beans at random.
3. What is the *variant* of the program you wrote just above?
4. Write a loop *invariant* that can be used to verify that your prediction (from the first question) is correct.

1. The parity of the number of black beans is an invariant. Hence, if we had an even number of black beans initially, the final bean will be white; if we had an odd number, it will be black.
2. This uses the encoding of **while** presented in the lectures. That is, **while** b **do** P **od** should be understood as shorthand for $(b; P)^*; \neg b$.

```
while ( $b + w > 1$ ) do
  ( $b > 1; b := b - 2; w := w + 1$ )
  + ( $b > 0 \wedge w > 0; w := w - 1$ )
  + ( $w > 1; w := w - 1$ )
od
```

Each loop iteration is a non-deterministic choice between drawing two black beans, two different beans, or two white beans. Each choice is guarded by a check that makes sure we have enough beans to actually make this draw.

3. The variant is $b + w$.

4. An invariant is:

$$b \bmod 2 = N \wedge b \geq 0 \wedge w \geq 0$$

where N is an integer constant that doesn't depend on any of the program variables.

You can also mention the specific parity, which means you'd technically need two separate Hoare logic proofs for the even and odd cases, respectively.

$$\text{odd}(b) \wedge b \geq 0 \wedge w \geq 0$$

Extending our Language

Suppose we wanted to extend $\square$ with a "repeat" loop

do s until b

that runs the loop body s at least once: it checks the guard of the loop only *after* each iteration of the loop body, exiting the loop if the guard is true.

1. Propose a translation of this repeat loop into existing $\square$ constructs.
2. Validate your translation by showing that the following Hoare logic rule for the repeat loop is derivable for your translation:

$$\frac{\{\phi\} s \{\phi\}}{\{\phi\} \text{do } s \text{ until } e \{\phi \wedge e\}} \quad .$$

1. s ; **while**($\neg b$) **do** s **od**

2. This proof tree elides the rule names, as well as a couple of implications of the form $A \rightarrow A$ from the rule of consequence:

$$\frac{\{\phi\} s \{\phi\} \quad \frac{\frac{\phi \wedge \neg b \rightarrow \phi \quad \{\phi\} s \{\phi\}}{\{\phi \wedge \neg b\} s \{\phi\}} \quad \frac{\{\phi\} \text{while}(\neg b) \text{do } s \text{od } \{\phi \wedge \neg e\}}{\{\phi\} \text{while}(\neg b) \text{do } s \text{od } \{\phi \wedge e\}} \quad \phi \wedge \neg e \rightarrow \phi \wedge e}{\{\phi\} s; \text{while}(\neg b) \text{do } s \text{od } \{\phi \wedge e\}}$$

Program algebra

Program algebra is the manipulation of program *texts* in a way that preserves their meaning, and (hopefully) leads to a better or more understandable program than

before. An example is the equality between these two programs:

$$\llbracket x := 1; y := 2 \rrbracket = \llbracket y := 2; x := 1 \rrbracket$$

For these two programs, at least sequential composition commutes. But it is not the case, in general, that,

$$\llbracket P; Q \rrbracket = \llbracket Q; P \rrbracket$$

1. Find a counterexample to the above formula; or, in other words, show that sequential composition is not commutative.
2. Show that sequential composition is associative.
3. What is the identity element of sequential composition? Or in other words, what is a program P such that for all Q ,

$$\llbracket P; Q \rrbracket = \llbracket Q; P \rrbracket = \llbracket Q \rrbracket$$

4. Are the following algebraic laws true in general or not?

$$\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket P; \text{while } g \text{ do } P \text{ od} \rrbracket$$

$$\llbracket \text{if } g \text{ then } P; Q \text{ else } P; R \text{ fi} \rrbracket = \llbracket P; \text{if } g \text{ then } Q \text{ else } R \text{ fi} \rrbracket$$

$$\llbracket \text{while } g \text{ do } (\text{while } g \text{ do } P \text{ od}) \text{ od} \rrbracket = \llbracket \text{while } g \text{ do } P \text{ od} \rrbracket$$

$$\llbracket \text{while } g \text{ do } (\text{while } g' \text{ do } P \text{ od}) \text{ od} \rrbracket = \llbracket \text{while } g \wedge g' \text{ do } P \text{ od} \rrbracket$$

1. For example, let $P = x := 1$ and $Q = x := 2$. The final value of x will be different depending on which one happens last.

2. Step by step, as follows:

$$\begin{aligned} \llbracket (P; Q); R \rrbracket &= \llbracket P; Q \rrbracket; \llbracket R \rrbracket && \text{(definition)} \\ &= (\llbracket P \rrbracket; \llbracket Q \rrbracket); \llbracket R \rrbracket && \text{(definition)} \\ &= \llbracket P \rrbracket; (\llbracket Q \rrbracket; \llbracket R \rrbracket) && \text{(assoc. of relational composition)} \\ &= \llbracket P \rrbracket; \llbracket Q; R \rrbracket && \text{(definition)} \\ &= \llbracket P; (Q; R) \rrbracket && \text{(definition)} \end{aligned}$$

3. The identity element of sequential composition is **skip**. The semantics of **skip** is the identity relation, so the identity laws follow from the fact that the identity relation is the left and right identity of relational composition.

4. As follows:

- It is *not* the case that

$$\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket P; \text{while } g \text{ do } P \text{ od} \rrbracket$$

because the two exhibit different behaviour if g is false initially.

- It does *not* hold that

$$\llbracket \text{if } g \text{ then } P; Q \text{ else } P; R \text{ fi} \rrbracket = \llbracket P; \text{if } g \text{ then } Q \text{ else } R \text{ fi} \rrbracket$$

because executing P may change the truth value of g . If P cannot change the truth value of g , it becomes provable.

- It does hold that

$$\llbracket \text{while } g \text{ do (while } g \text{ do } P \text{ od) od} \rrbracket = \llbracket \text{while } g \text{ do } P \text{ od} \rrbracket$$

Two key facts about the relational operators (not proved here) are used in its proof:

$$(R^*)^* = R^*$$

$$(\langle g \rangle; (\langle g \rangle; R)^*)^* = (\langle g \rangle; R)^*$$

- It is *not* the case that

$$\llbracket \text{while } g \text{ do (while } g' \text{ do } P \text{ od) od} \rrbracket = \llbracket \text{while } g \wedge g' \text{ do } P \text{ od} \rrbracket$$

Consider the case where $g = \top$ and $g' = \perp$. The former will loop forever, but the latter will not.